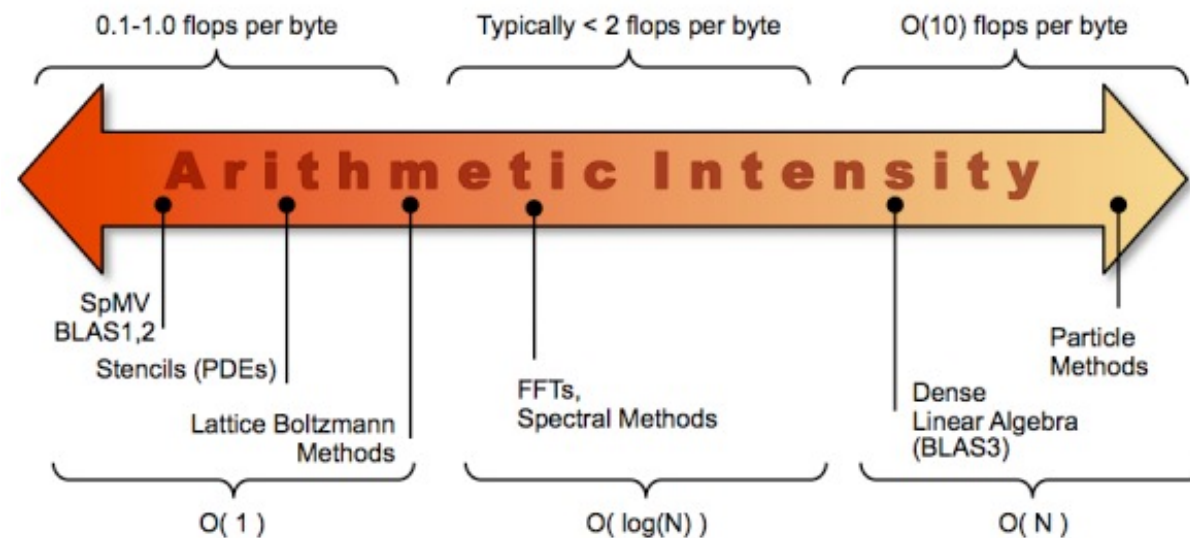


Parallel Algorithms for GPUs

Parallel Algorithms

João Luís Sobral (jls@di ...)

April/2026



Data storage

Regular : vector, matrix

Irregular (pointer based):
linked-list, tree, graphs

Data accesses

- Regular / irregular

Recap: GPU hardware & vector processing

- GPU similarities to **vector machines**:

- Work well with data-level parallel problems
- Scatter-gather transfers (note: to address lack of spatial data locality)
- Mask registers (note: to address control dependencies)
- **Large register files**



Hardware-based thread scheduling:

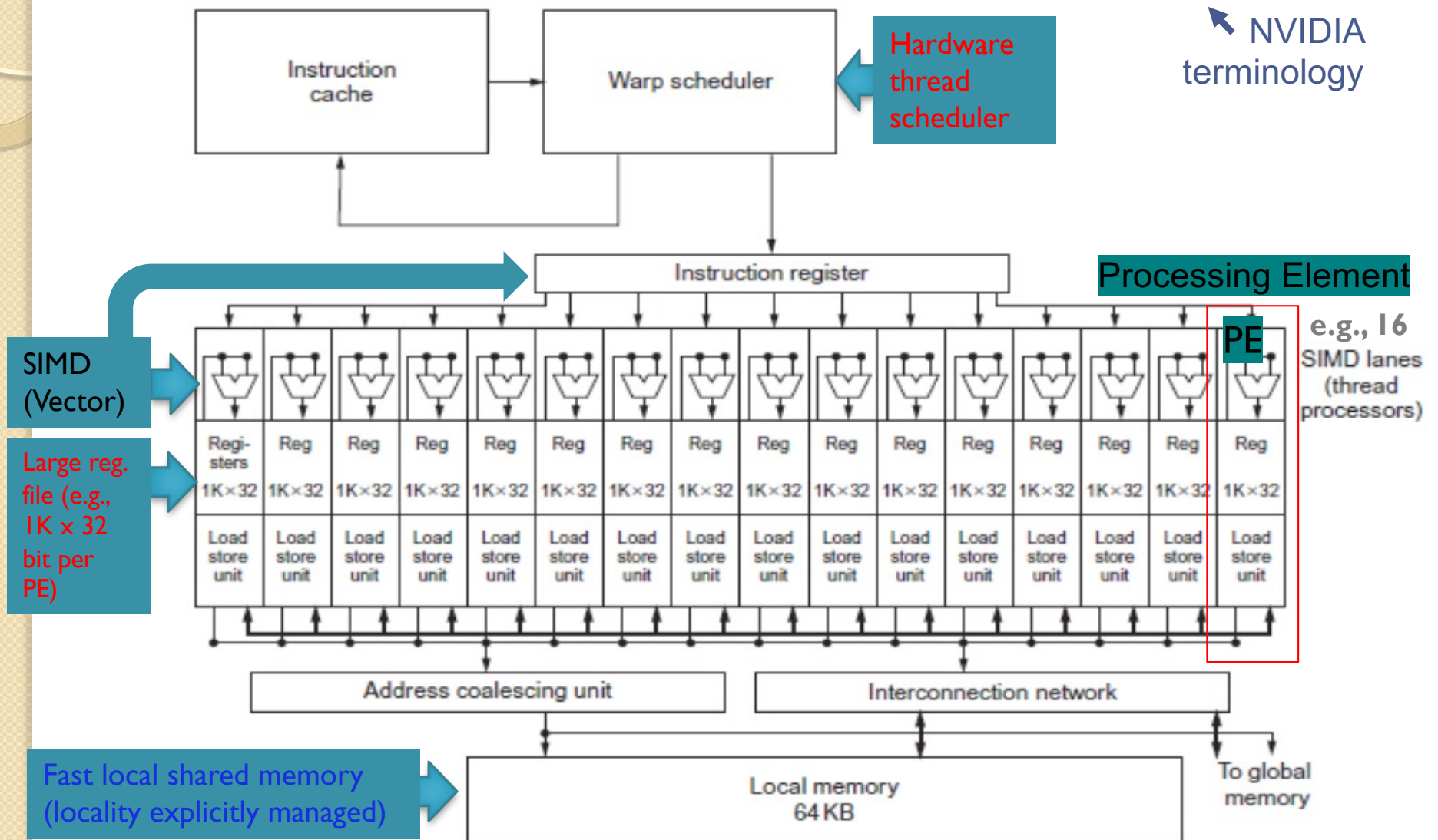
Registers are allocated to threads at creation
+ fast thread context switch (& lower cost)
- limits the maximum number of threads per Core

- Differences:

- No scalar processor
- **Uses multithreading to hide [memory] latency**
 - **Wide and shallow compared to vector processors:**
 - Threads are organized into (wide) blocks (1-1024 threads per block)
 - Shallow pipeline

Recap: GPU Organization (NVIDIA)

A Multithreaded SIMD Processor (SM - Streaming Multiprocessor)



GK110 Kepler Architecture (K20 GPU on our cluster)

Kepler: 15 SM, 2880 PEs **October'13**

Each SM is like a PU on a multicore processor

SMs internally rely on vector processing

Each SM contains (GK110):

- 192 PEs FP32 (green PEs)
- 64 PEs FP64 (yellow PEs)
- 32 Load/store PEs (grey PEs)
- 32 PEs for special functions



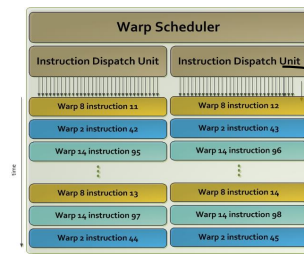
Quad Warp Scheduler

SIMD lanes

Quad Warp Scheduler (Wide and shallow)

The SM schedules threads in **groups of 32 parallel threads** called warps. Each SM features four warp schedulers and eight instruction dispatch units, allowing four warps to be issued and executed concurrently. **Kepler's quad warp scheduler selects four warps, and two independent SIMD instructions per warp can be dispatched each cycle.**

wide (32 SIMD lanes)



SMX: 192 single-precision CUDA cores, 64 double-precision units, 32 special function units (SFU), and 32 load/store units (LD/ST).

Recap: Vector Programming (& challenges)

- 
- 
- 
- 
- Compilers are a key element to give hints on whether a code section will vectorize or not
 - Check if loop iterations have **data dependencies** and/or if...then statements, otherwise vectorization is compromised
 - Vector architectures have a too high cost, but simpler variants are currently available on off-the-shelf devices, as SIMD extensions to the scalar PUs; **however**:
 - most do not support **non-unit stride**
=> care must be taken in the design of data structures (data layout)
 - same applies for **mask register**,
 - **gather-scatter...**
- No data dependencies allowed**
- Vector prog. applies only to data-parallelism (e.g. map pattern)
- Deal with control dependencies**
- common solutions:
 - predication => additional work
 - mask registers => unused PEs
- Support for arbitrary vector lengths**
- when the vector registers size does not match the vector length
- Maximize data locality**
- maximize spatial locality
 - data alignment in memory

Recap: CUDA optimization example: Reduction

- Recap: Common **data parallelism** patterns

- Map**: applies side-effect free unary function to all input elements

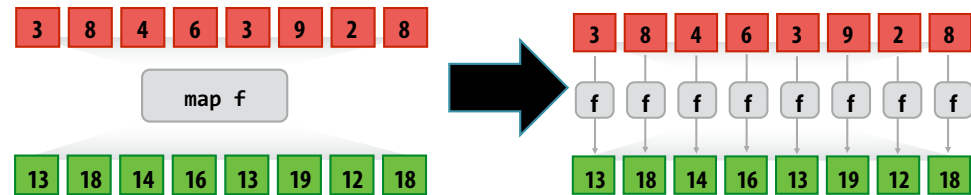
C++

```
int f(int x) { return x + 10; }
```

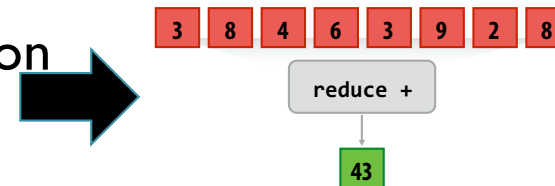
```
int a[] = {3, 8, 4, 6, 3, 9, 2, 8};
```

```
int b[8];
```

```
std::transform(a, a+8, b, f);
```



- Reduce**: Applies a binary operation returning an accumulated value



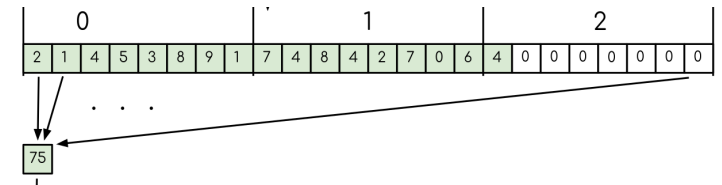
- Naive GPU reduce implementation** (for $\#threads < \#itens$)

Local “sequential” reduce of $\#itens/\#threads$ itens

& atomic operation on global memory (atomicAdd)

- Main problems:

- no spatial locality when reading global memory
- sequential access on final write to global memory (one atomic operation per thread!)



Recap: CUDA optimization example: Reduction

- Three **illustrative** improvements

1. Improve global **memory accesses**

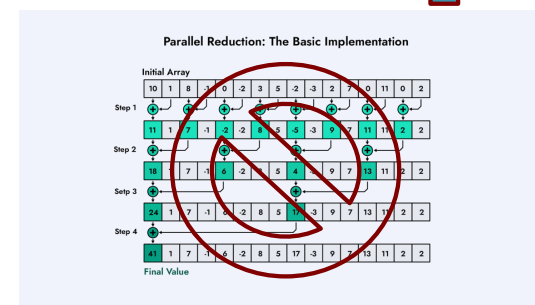
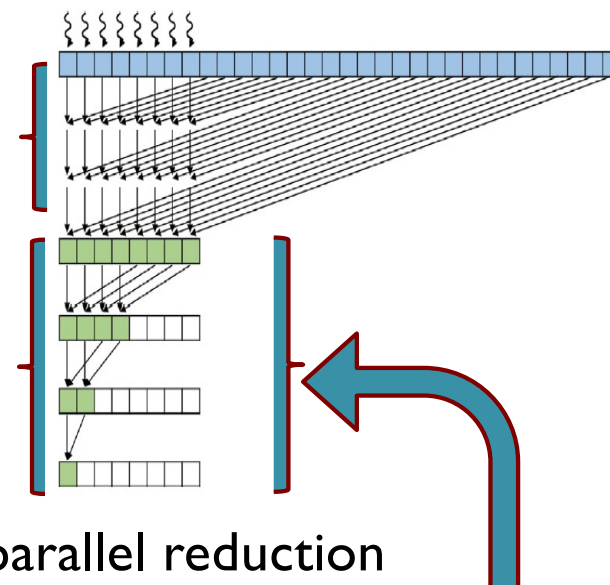
- **Spatial locality** (a.k.a. coalescing)

2. Local **parallel reduction** on **ShMem**

- Store intermediate results on **ShMem**
- One atomic operation per thread block

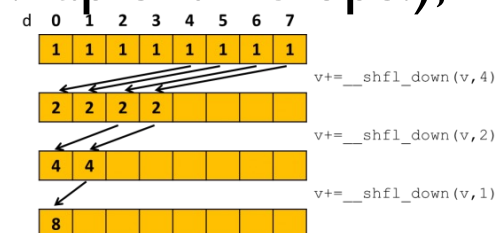
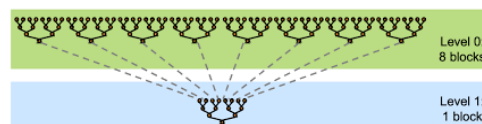
3. Minimize **control divergence** on local parallel reduction

```
for (unsigned int stride = blockDim.x; stride >= 1; stride /= 2) {
for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
    if (threadIdx.x < stride) if (threadIdx.x % stride == 0)
        sm_input[i] += sm_input[i + stride];
    __syncthreads(); // local thread block barrier
}
```



- **Other:** reduce **intra-block synchronisation** (wrap shuffle ops.),

multiple reduce **trees**
(task graphs?)



Parallel Algorithms Type I: regular algorithms (e.g., Monte Carlo / Matrix Multiplication / MD / Stencil)

- Main optimizations:
 - **Memory accesses coalescing**: improve spatial locality **intra-wrap**
 - **Use of shared memory**: explore temporal locality **intra-thread blocks**

- Examples:

- Matrix multiplication

```
#pragma omp parallel for
for(int i=0; i<N; i++)
  for(int k=0; k<N; k++)
    for(int j=0; j<N; j++)
      C[i][j] += A[i][k] * B[k][j];
```



```
for(int i=0; i<N; i++)
  for(int k=0; k<N; k++)
    #pragma omp parallel for
    for(int j=0; j<N; j++)
      C[i][j] += A[i][k] * B[k][j];
```

Intra-wrap coalesced memory reads

Intra-thread block (w/explicit packing)

- Stencil 2D

```
#pragma omp parallel for
for(int i=1; i<size-1; i++)
  for(int j=1; j<size-1; j++)
    stencil(a,aux,i,j);
```



```
for(int i=1; i<size-1; i++)
  #pragma omp parallel for
  for(int j=1; j<size-1; j++)
    stencil(localSharedA,aux,i,j);
```



Minimize **control divergence**

```
for(int i=1; i<size-1; i++)
  #pragma omp parallel for
  for(int j=0; j<size; j++)
    stencil(localSharedA,aux,i,j);
```


Parallel Algorithms Type II: non regular (e.g., Sorting)

- Best (GhatGPT) Radix sort LSD:

- Process the keys **digit by digit**
- Extract the current digit (bit range)
- Count how many elements fall into each bucket
- Compute prefix sums (scan)
- Scatter elements into correct positions
- Repeat for next digit



```
for (int shift = 0; shift < 32; shift += BITS_PER_PASS) {
    // 1. Compute histogram per block
    histogram<<<...>>>(input, ...);

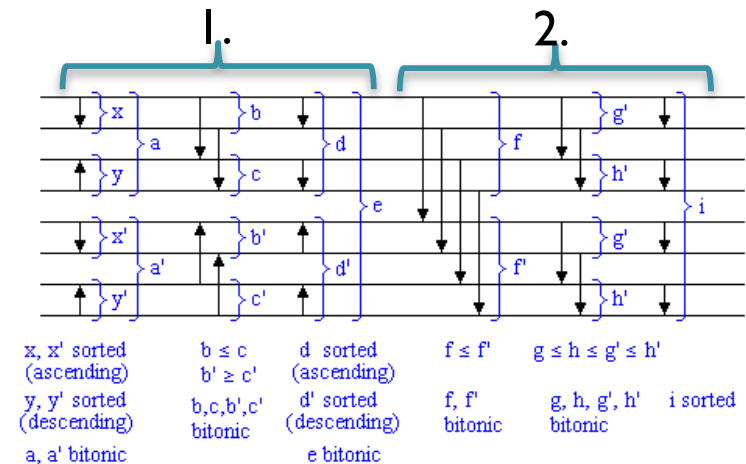
    // 2. Scan histograms → global offsets
    scan<<<...>>>(histogram, offsets);

    // 3. Scatter elements into output
    scatter<<<...>>>(input, output, offsets, shift);

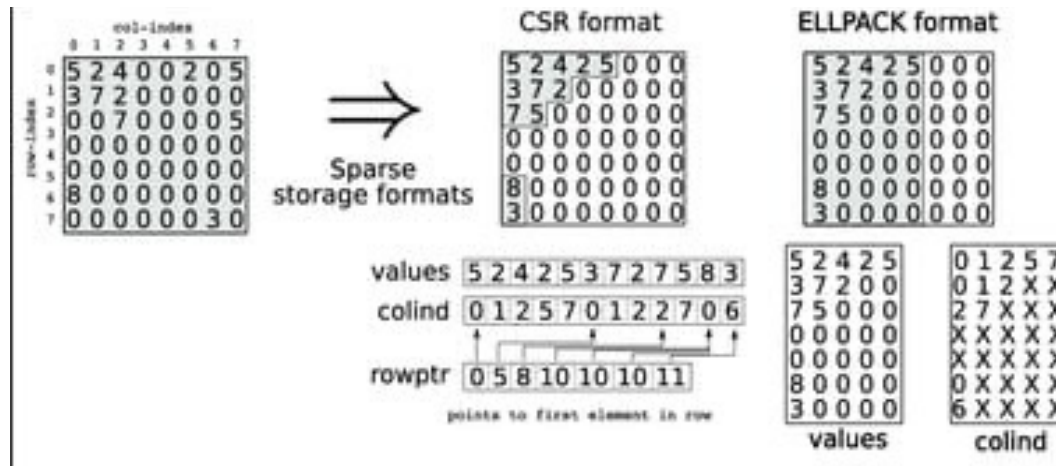
    // 4. Swap buffers
    swap(input, output);
}
```

- Classical (simpler: Bitonic Sort):

- **1. build a bitonic sequence**
(e.g. [1, 3, 7, 14, 12, 8, 5, 2])
- **2. bitonic merge** of sequences
 - Compare elements at a specific distance and swap if out of order
 - Recursively apply the process to both halves until fully sorted.



- GPU SpMV implementations?



- 

```
#pragma omp parallel for
for (int r = 0; r < number_rows; r++) {
    y[r] = 0.0;
    for (int j = rowptr[r]; j < rowptr[r + 1]; j++)
        y[r] += values[j] * x[colind[j]]
}
```

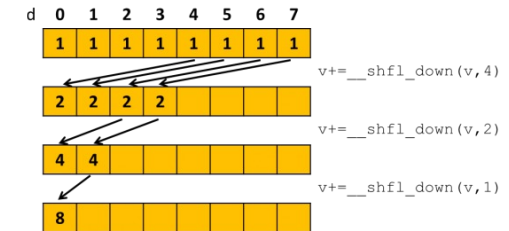
-

```
for (int r = 0; r < number_rows; r++) {
    y[r] = 0.0;
    #pragma omp parallel for
    for (int j = rowptr[r]; j < rowptr[r + 1]; j++)
        y[r] += values[j] * x[colind[j]]
}
```

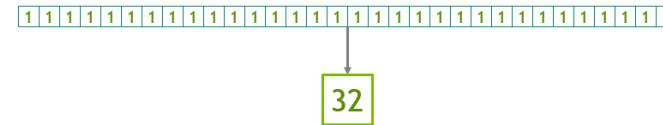
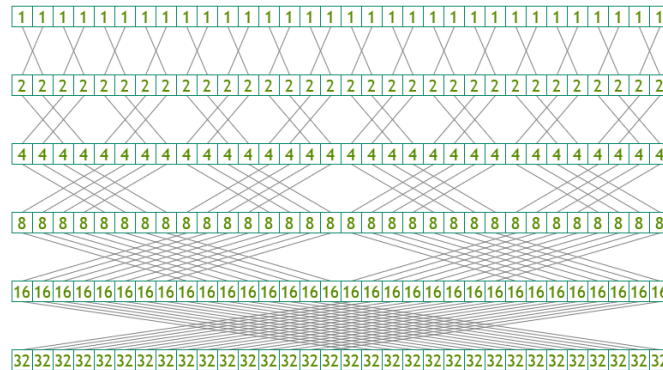
- colind has a fixed width! (multiple of wrap size?)

AI00 Improvements (Ampere GPU)

```
for (unsigned int stride = blockDim.x; stride >= 1; stride /= 2) {
for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
    if (threadIdx.x < stride) if (threadIdx.x % stride == 0)
        sm_input[i] += sm_input[i + stride];
    __syncthreads(); // local thread block barrier
}
```



WARP-WIDE REDUCTION IN A SINGLE STEP



```
__device__ int reduce(int value) {
    value += __shfl_xor_sync(0xFFFFFFFF, value, 1);
    value += __shfl_xor_sync(0xFFFFFFFF, value, 2);
    value += __shfl_xor_sync(0xFFFFFFFF, value, 4);
    value += __shfl_xor_sync(0xFFFFFFFF, value, 8);
    value += __shfl_xor_sync(0xFFFFFFFF, value, 16);

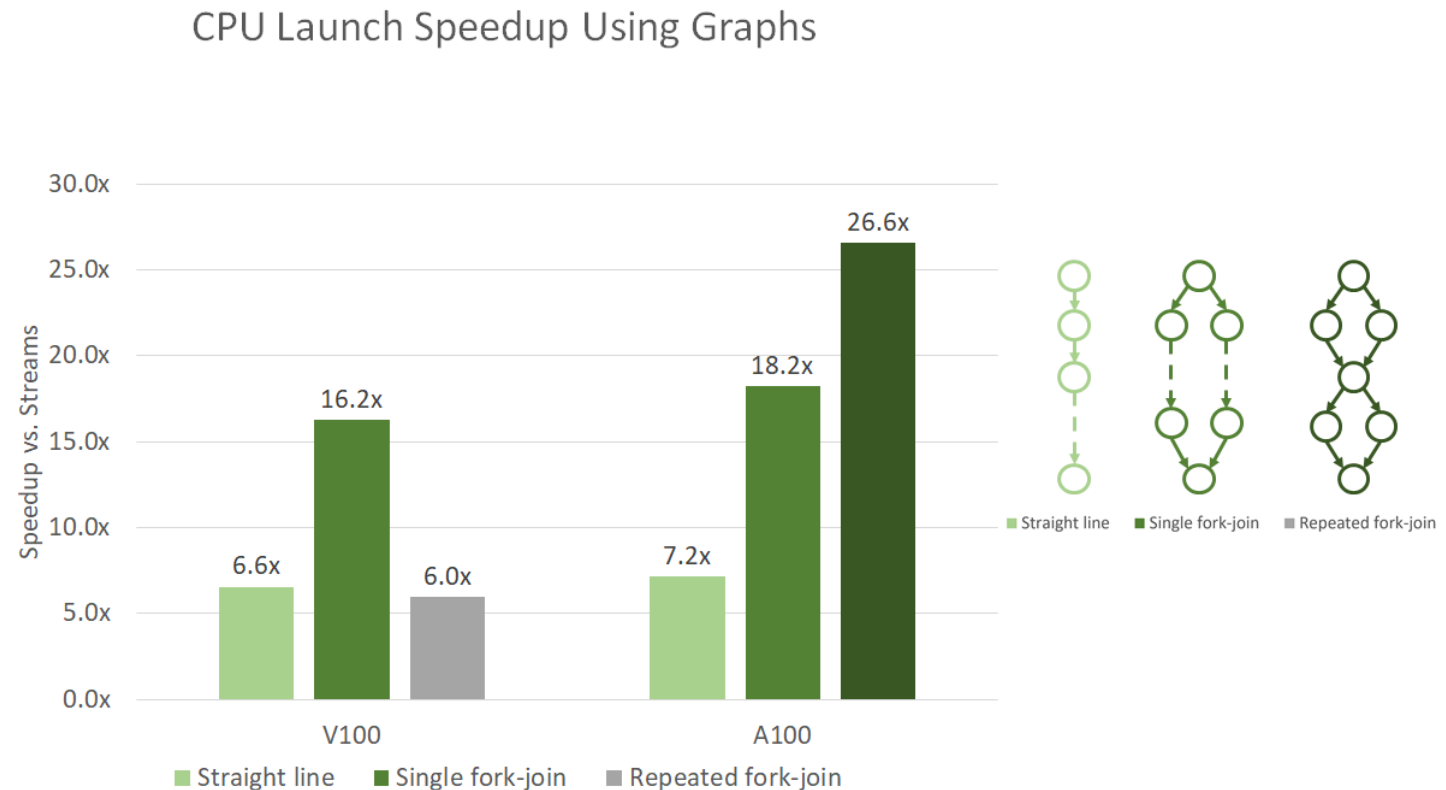
    return value;
}
```

```
int total = __reduce_add_sync(0xFFFFFFFF, value);
```

```
thread_block_tile<32> tile32 =
    tiled_partition<32>(this_thread_block());

// Works on all GPUs back to Kepler
cg::reduce(value, tile32, cg::plus<int>());
```

A100 Improvements (Ampere GPU)



Comparing CPU cost of whole-graph launch vs. the equivalent time to launch without graphs. Graph launch provides a significant boost on any hardware, but A100's launch optimizations provide a significant gain, in particular for graphs with more complex topologies.

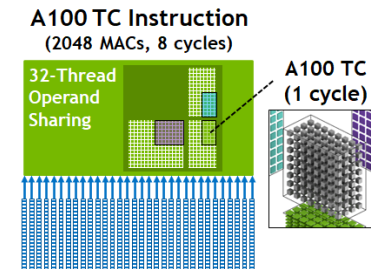
Figure 29. Impact of Task Graph acceleration on CPU launch latency

AI00 Improvements (Ampere GPU)

- Tensor Core (TC) improvements (3rd Gen)

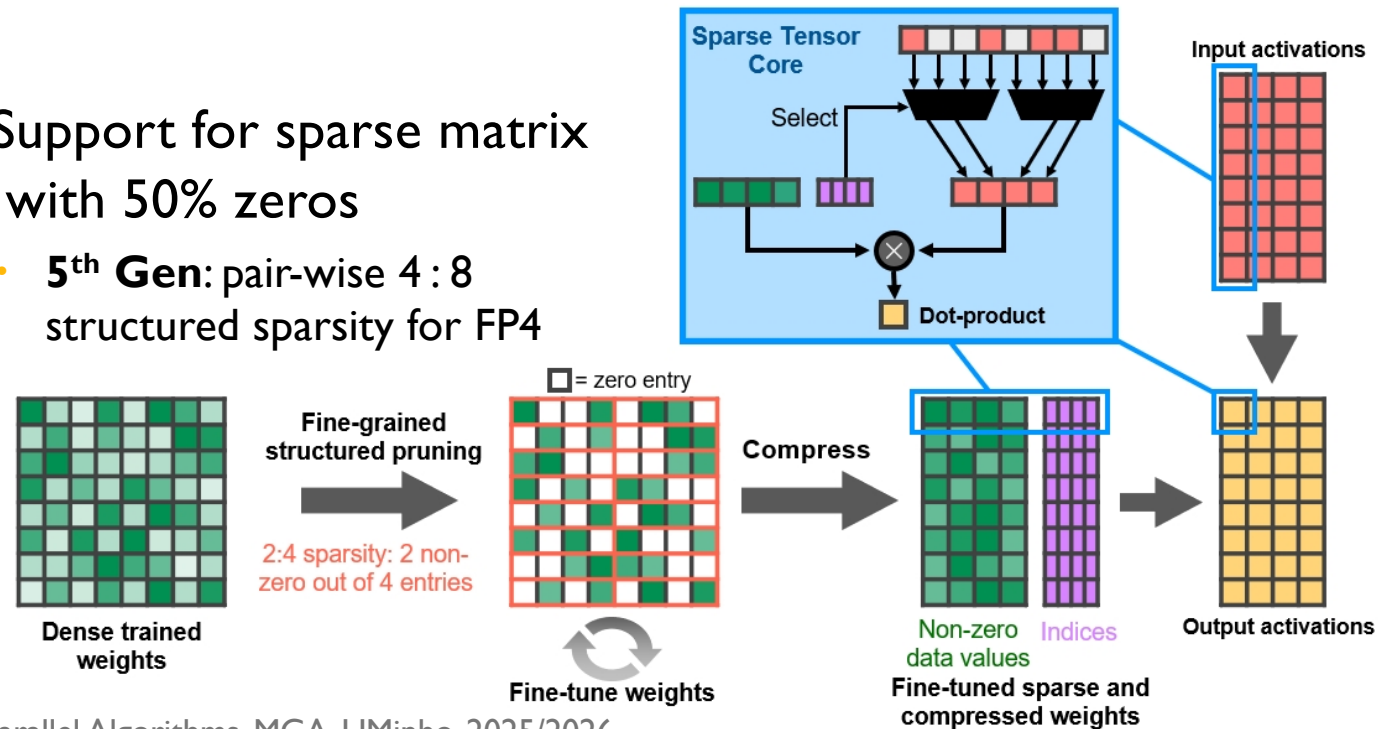
- Single instruction for a 16x8x16 matrix multiplication (8 cycles)

- 4th Gen: warpgroup-level MMA
- 5th Gen: TMEM (128x512x32 bits)



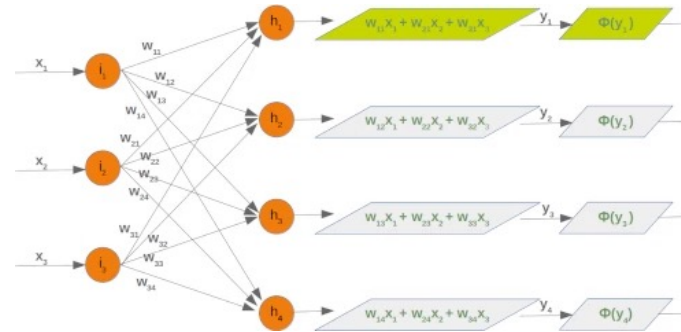
- Support for sparse matrix with 50% zeros

- 5th Gen: pair-wise 4 : 8 structured sparsity for FP4



Crosscut

- DNN as a SpMV



- FVLib as SpMV?
- Data types supported

Data Type Precision Reduction						
Architecture	FP64	F16 / FP32	INT8 / INT32	INT4 / INT32	F8 / FP32	MXFP / FP32
Volta	X	FP16	X	X	X	X
Turing	X	FP16	V	V	X	X
Ampere	V	FP16 / BF16	V	V	X	X
Hopper	V	FP16 / BF16	V	V ³	FP8 / FP22 ¹	X
Blackwell	V	FP16 / BF16	V	X	V	MXFP8 MXFP6 MXFP4 NVFP4
Blackwell Ultra	V ²	FP16 / BF16	V ²	X	V	MXFP8 MXFP6 MXFP4 NVFP4

Notes: (1) Effective precision. (2) Massively reduced throughput. (3) Only supported on only on legacy mma.sync.